Privacy Preserving Link Prediction

Kaleb Kim, Wiam Skakri, Carson Whitehouse, Jonah Lorenzo, Kent Manion, Aidan Bugayong

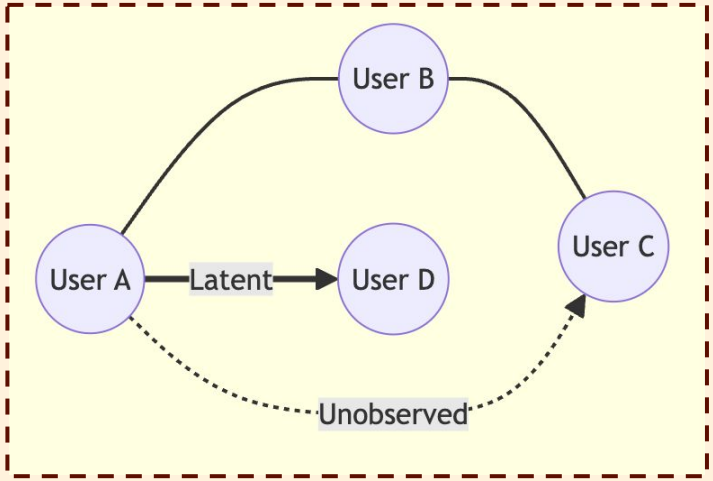
Goal

To build a Python library that allows anyone to apply **privacy preserving link prediction**.

Refresher:
Link Prediction

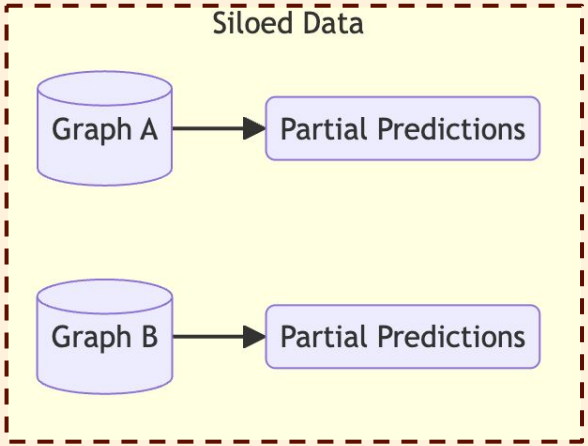
Link Prediction

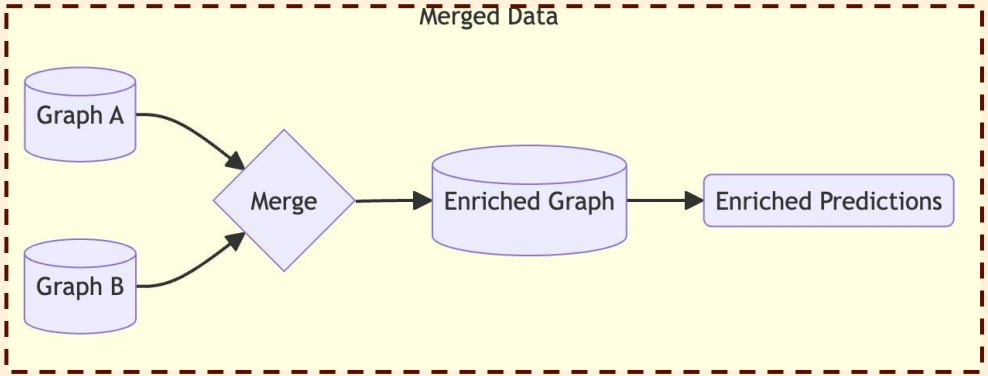
- Link prediction helps discover unobserved or latent connections between nodes in a graph
- Data holders rank the likelihood of new connections forming overtime
- Useful for social networks, e-commerce, telecomms, bioinformatics…



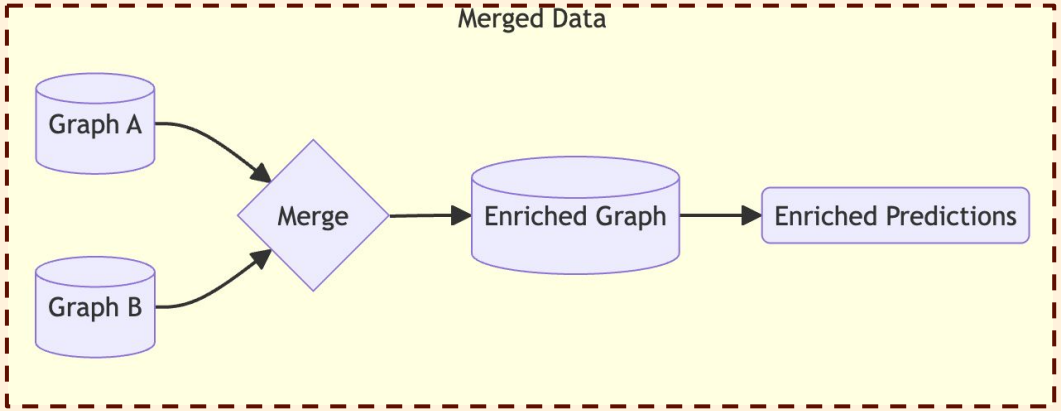
Link Prediction

- Link prediction is typically done on a single local graph
- Link prediction can be more accurate if we merge two or more graph databases that include similar information (Distributed Link Prediction)





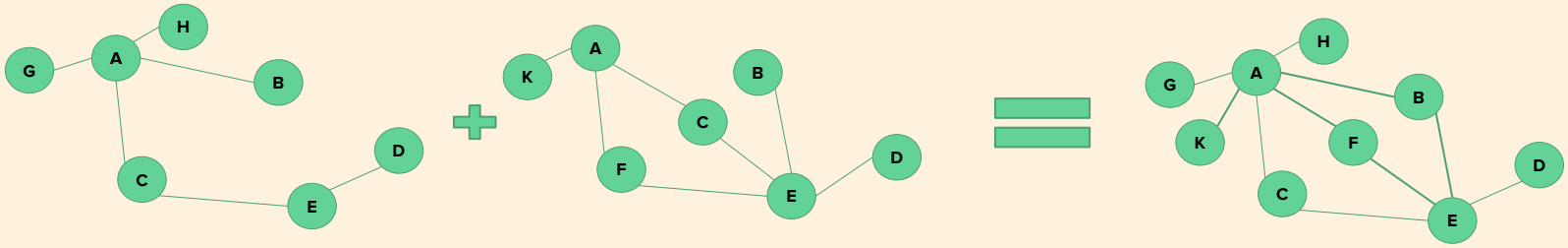
Distributed Link Prediction



- Two parties can utilize the connections in their combined graph to provide more accurate link predictions
- In other cases, distributed link prediction allows link prediction between nodes based on other attributes

Distributed Link Prediction

- In some cases, collaboration is mutually beneficial to contributing parties
- In other cases, the second party is paid to participate
- Distributed Link Prediction results in privacy concerns since it implies combining two or more different graph databases!
  - Identity Disclosure
  - Link Disclosure
  - Attribute Disclosure



Refresher: Privacy Preserving Link Prediction

Privacy preserving link prediction allows multiple data holders to collaboratively forecast unobserved or latent connections between nodes **without explicitly revealing what each party knows.**

Use Cases I

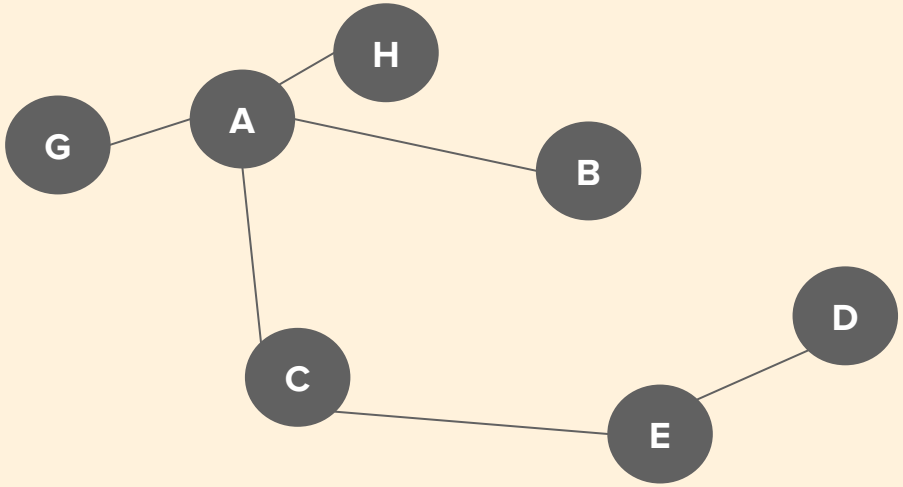
- **Social Networks:** Privacy-First recommendations
  - Discovering friend suggestions between users based on structural similarity (Think of # Mutual Connections on Instagram!)
  - Does not expose the full contact list of either party
- **Telecommunications:** Targeted Advertising
  - Finding target audience members similar to an existing set
  - Link prediction on phone networks while keeping the actual call/link records private
- **E-Commerce:** Collaborative Filtering
  - Recommending products to a user based on similarities between them and others' purchasing
  - Does not use a central server seeing individual transaction histories

Use Cases II

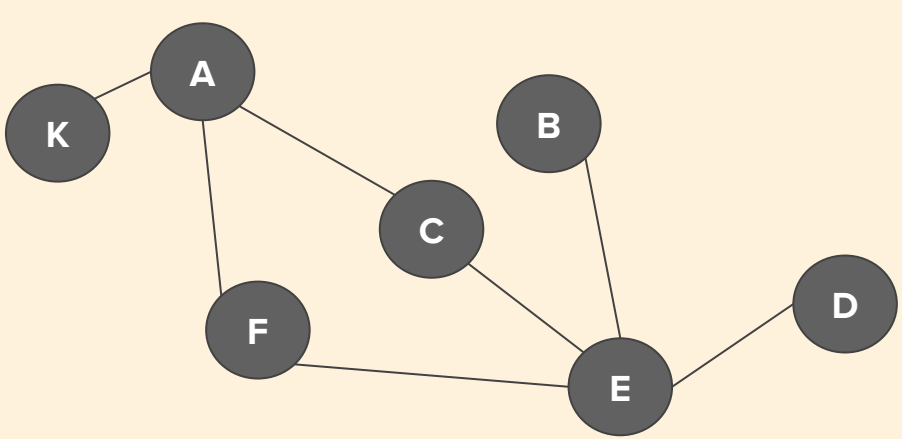
- **Bioinformatics:** Secure EHR Research
  - Merging Electronic Health Records (EHR) with clustering graphs to predict disease spread or patient diagnoses
  - Does not reveal sensitive protected health information (PHI).
- **Financial Fraud Detection:** Exposing multibank criminal networks
  - PPLP supports fraud detection by identifying suspicious latent connections between accounts at different institutions which is a marker for money-laundering
  - Raw graph sharing is prohibited by financial privacy regulation, so this could allows banks observe the transaction subgraph of a potentially criminal organization.

Revisiting our Example

Subset only Looking N(A,E)

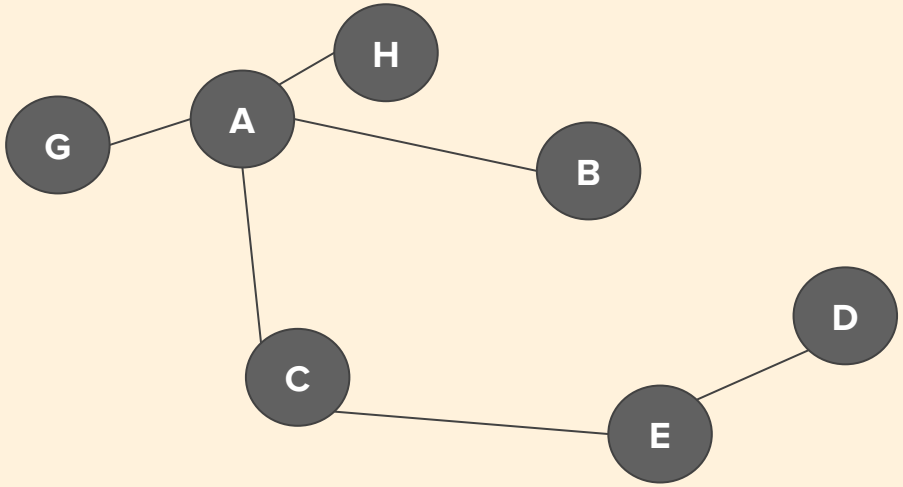


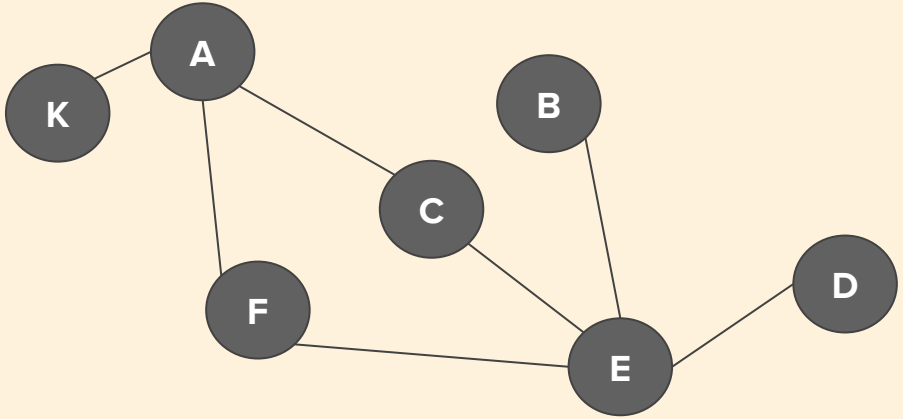
Alice



Bob

Searching Local Graph





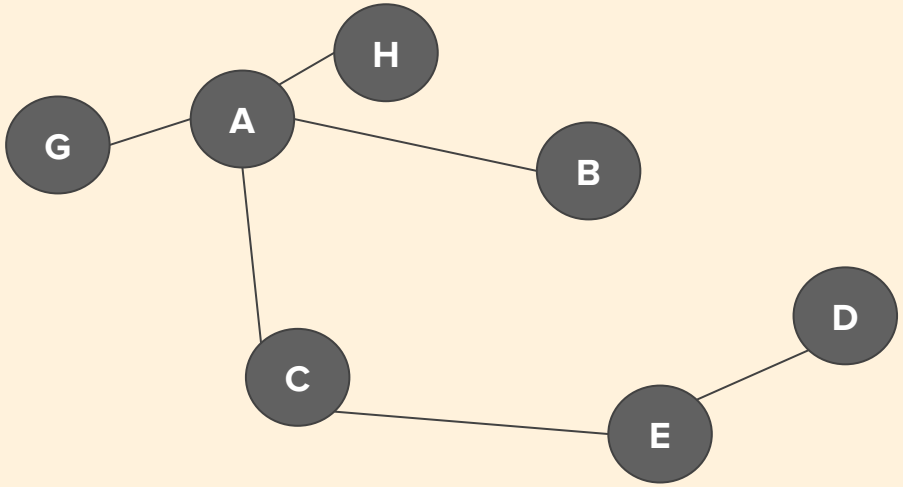
Alice

$N_A(A)=\{H,G,B,C\}$
$N_A(E)=\{D,C\}$
$N_A(A) \cap N_A(E) = I_A(A,E) = \{C\}$

$N_B(A)=\{K,F,C\}$
$N_B(E)=\{B,C,F,D\}$
$N_B(A) \cap N_B(E) = I_B(A,E) = \{C,F\}$

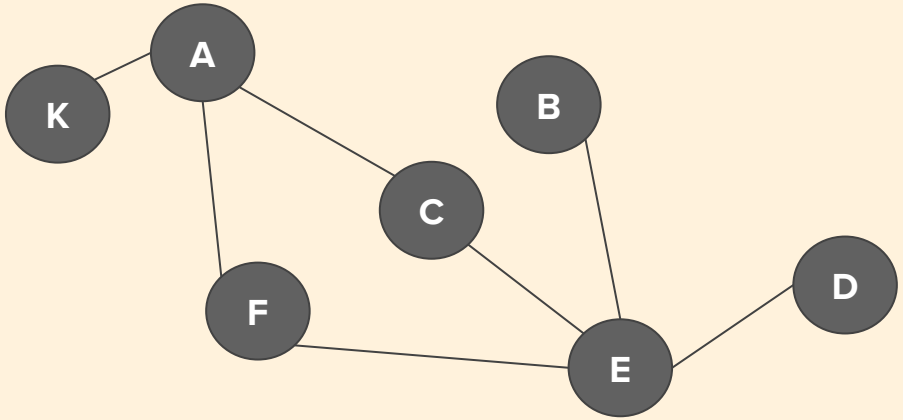
Bob

Searching Local Graph



Alice

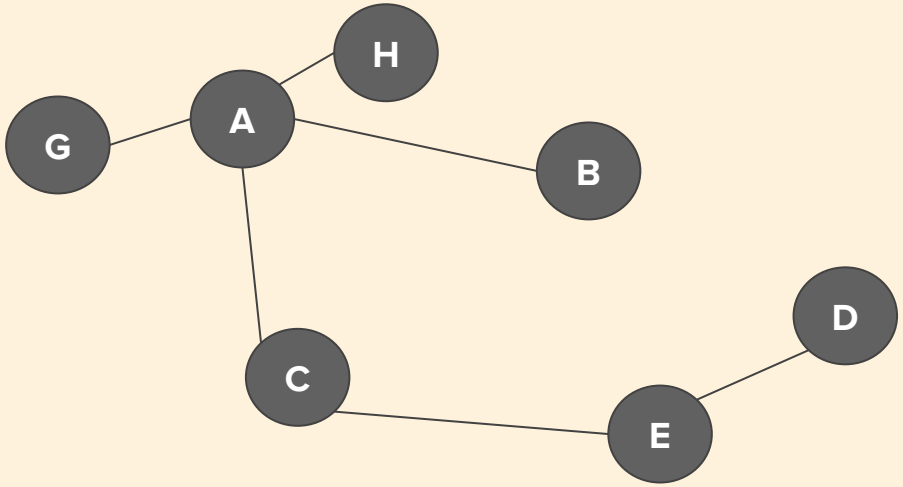
$N_A(A)=\{H,G,B,C\}$
$N_A(E)=\{D,C\}$
$N_A(A) \cap N_A(E) = I_A(A,E) = \{C\}$
$N_A(A) - I_A(A,E) = \{H,G,B\}$
$N_A(E) - I_A(A,E) = \{D\}$

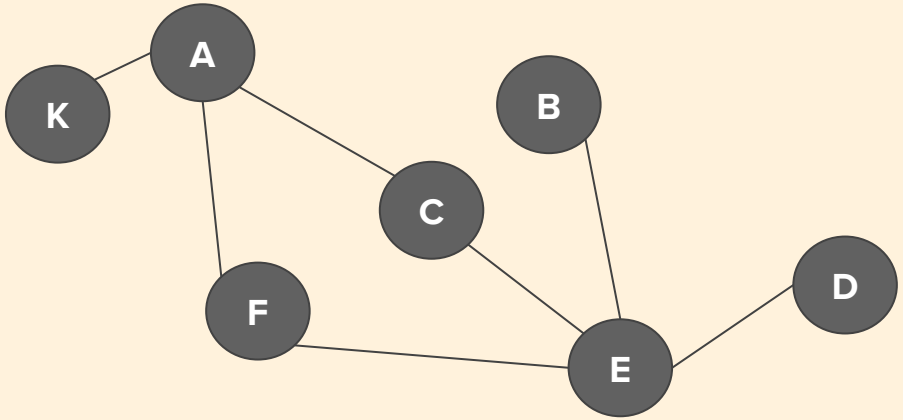


$N_B(A)=\{K,F,C\}$
$N_B(E)=\{B,C,F,D\}$
$N_B(A) \cap N_B(E) = I_B(A,E) = \{C,F\}$
$N_B(E) - I_B(A,E) = \{B,D\}$
$N_B(A) - I_B(A,E) = \{K\}$

Bob

Applying PSI





$N_A(A) \cap N_A(E) = I_A(A,E) = \{C\}$
$N_A(A-E)=\{H,G,B\}$
$N_A(E-A)=\{D\}$

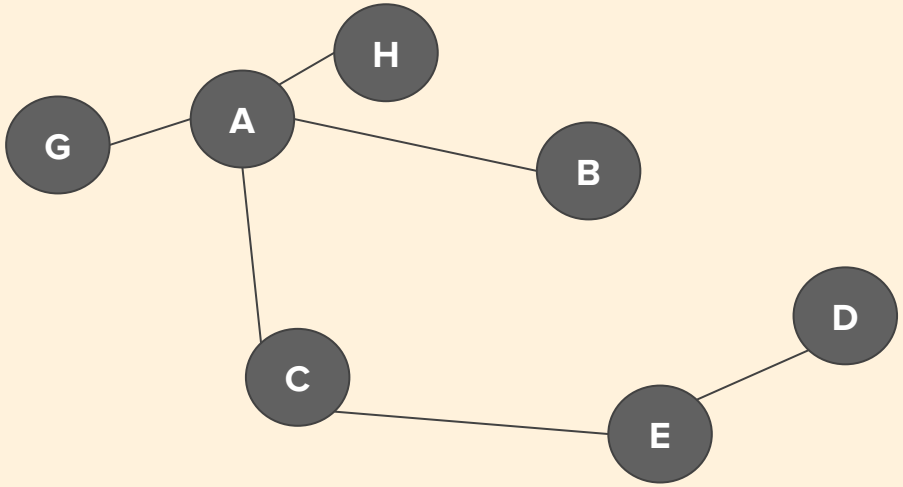
$N_B(A) \cap N_B(E) = I_B(A,E) = \{C,F\}$
$N_B(E-A)=\{B,D\}$
$N_B(A-E) =\{K\}$

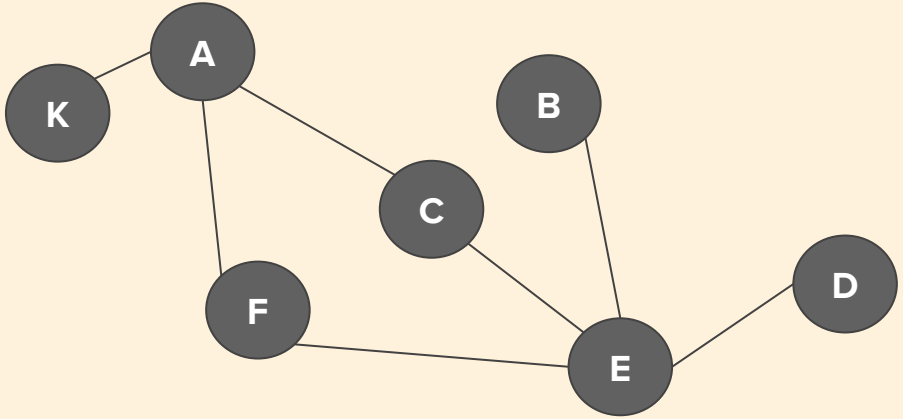
Alice

Bob

PSI
$I_A \cap I_B = I_{A+B} = \{C\}$

Applying PSI





$N_A(A) \cap N_A(E) = I_A(A,E) = \{C\}$
$N_A(A-E)=\{H,G,B\}$
$N_A(E-A)=\{D\}$

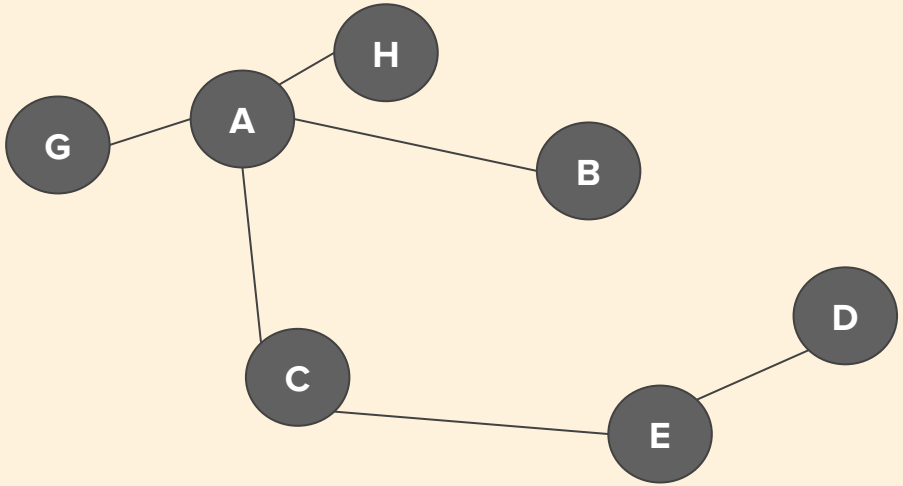
$N_B(A) \cap N_B(E) = I_B(A,E) = \{C,F\}$
$N_B(E-A)=\{B,D\}$
$N_B(A-E) =\{K\}$

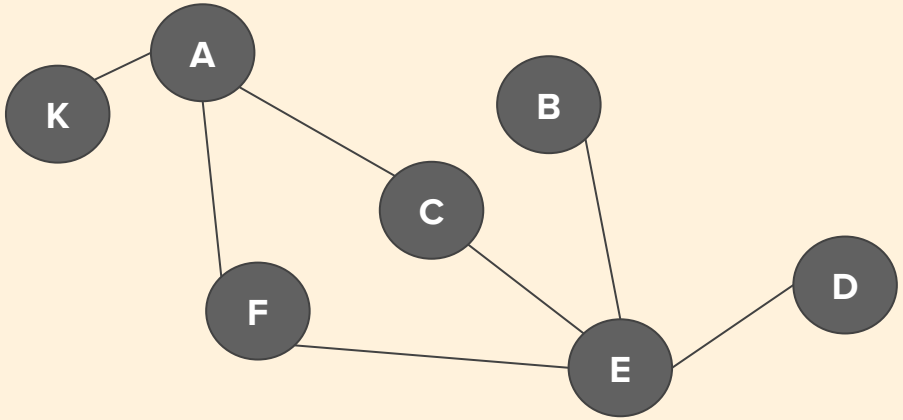
Alice

PSI
$I_A \cap I_B = I_{A+B} = \{C\}$
$N_A(A-E) \cap N_B(E-A) = \{B\}$

Bob

Applying PSI





$N_A(A) \cap N_A(E) = I_A(A,E) = \{C\}$
$N_A(A-E)=\{H,G,B\}$
$N_A(E-A)=\{D\}$

$N_B(A) \cap N_B(E) = I_B(A,E) = \{C,F\}$
$N_B(E-A)=\{B,D\}$
$N_B(A-E) =\{K\}$

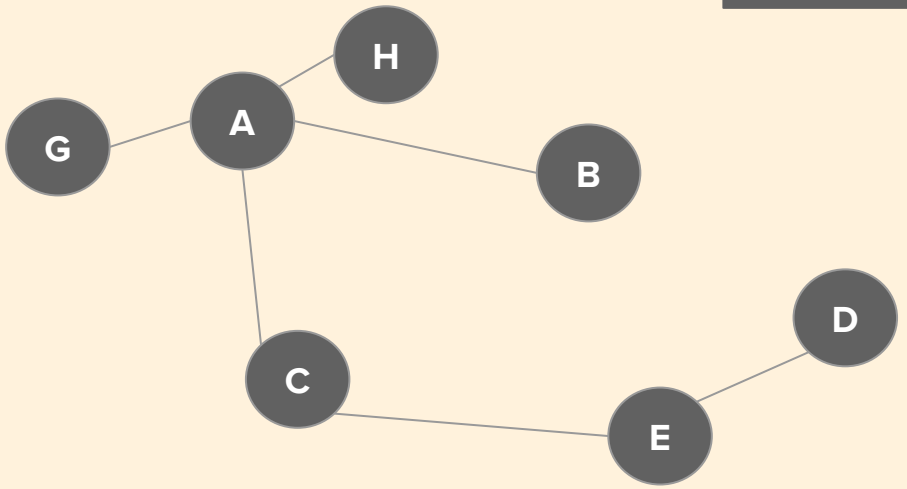
Alice

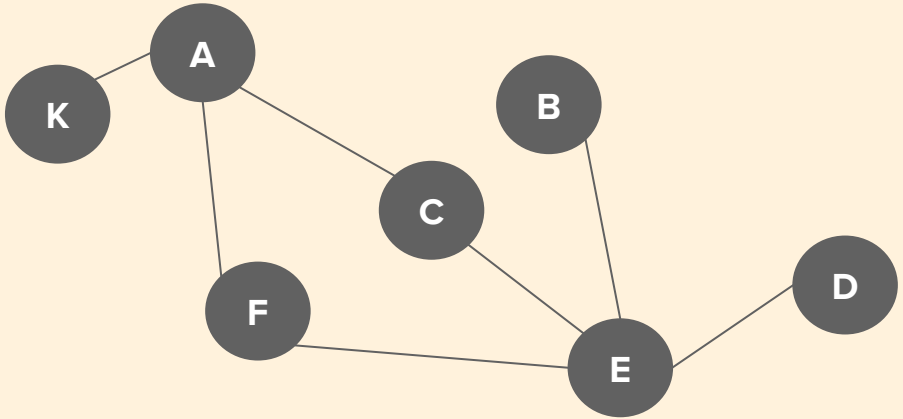
Bob

PSI
$I_A \cap I_B = I_{A+B} = \{C\}$
$N_A(A-E) \cap N_B(E-A) = \{B\}$
$N_A(E-A) \cap N_B(A-E) = \{\phi\}$

Size of CN(A,E)

$|CN(A,E)| = |I_A| + |I_B| - |I_{A+B}| + |\{B\}| + |\{\phi\}| = 3$





Alice

$N_A(A)=\{H,G,B,C\}$
$N_A(E)=\{D,C\}$
$I_A(A,E) = \{C\}$

$N_B(A)=\{K,F,C\}$
$N_B(E)=\{B,C,F,D\}$
$I_B(A,E) = \{C,F\}$

Bob

PSI

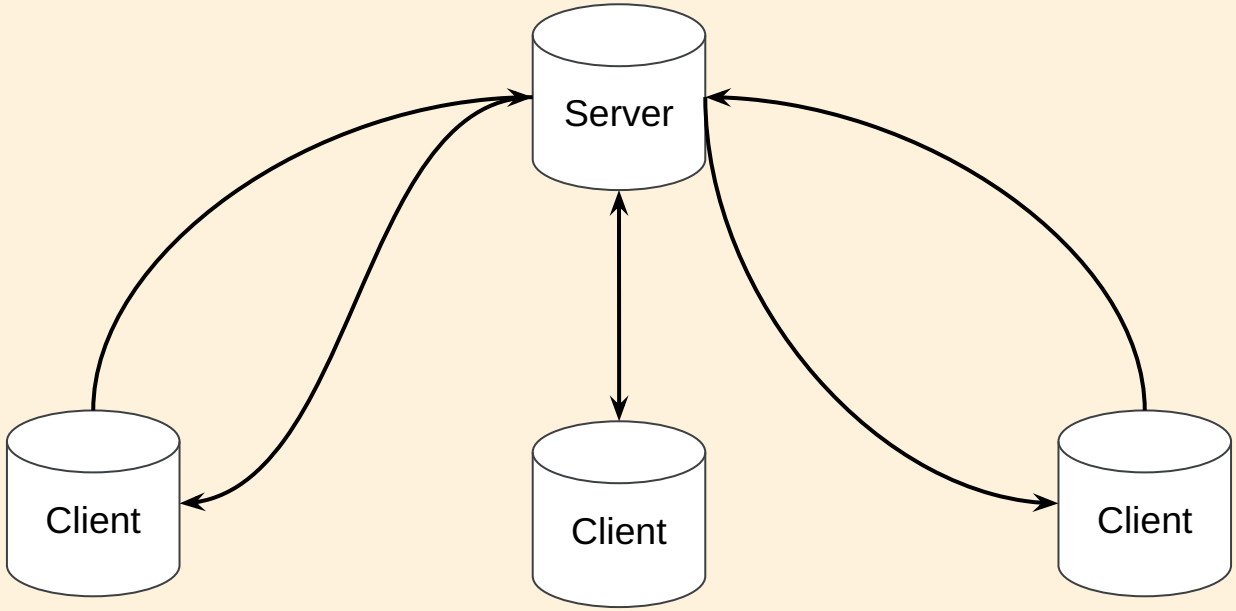
$I_A \cap I_B = I_{A+B} = \{C\}$

$N_A(A-E) \cap N_B(E-A) = \{B\}$

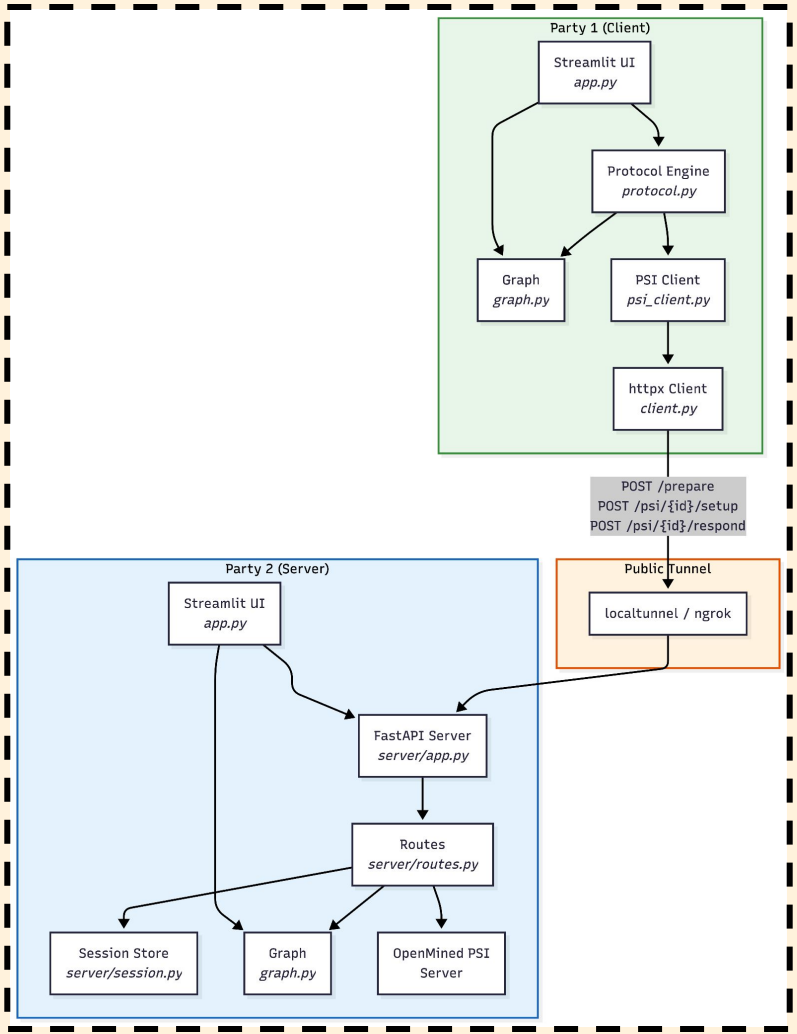
$N_A(E-A) \cap N_B(A-E) = \{\phi\}$

Project Details

System Design Architecture



- Owner hosts a running PPLP library instance, and users connect with a shared link
- Practically, can monetize with a “Pay-Per-Query" business model
  - I.e., users run secure queries for a service fee


Party 1 (Client)
Streamlit UI
app.py
Protocol Engine
protocol.py
Graph
graph.py
PSI Client
psi_client.py
httpx Client
client.py
POST /prepare
POST /psi/{id}/setup
POST /psi/{id}/respond
Public Tunnel
localtunnel / ngrok
Party 2 (Server)
Streamlit UI
app.py
FastAPI Server
server/app.py
Routes
server/routes.py
Session Store
server/session.py
Graph
graph.py
OpenMined PSI
Server

Demo

Server - Online Social Network Example (With Familiar Names)

Erman_Ayday,Yuqiao_Xu
Erman_Ayday,Wiam_Skakri
Erman_Ayday,Kaleb_Kim
Wiam_Skakri,Tahir_Shah
Wiam_Skakri,Kent_Manion
Wiam_Skakri,Aidan_Bugayong
Kaleb_Kim,Zeien_Chase
Kaleb_Kim,Jonah_Lorenzo
Kaleb_Kim,Kent_Manion
Kent_Manion,Zeien_Chase
Kent_Manion,Carson_Whitehouse
Carson_Whitehouse,John_Yoon
Carson_Whitehouse,Jonah_Lorenzo
Jonah_Lorenzo,Nathan_Link
Jonah_Lorenzo,Aidan_Bugayong
Aidan_Bugayong,Logan_Camp
Aidan_Bugayong,Kent_Manion
Nathan_Link,Zeien_Chase
Yuqiao_Xu,John_Yoon
Erman_Ayday,Aidan_Bugayong
Kaleb_Kim,Wiam_Skakri
Carson_Whitehouse,Aidan_Bugayong
Darin_Hall,Eric_Kaler

Client1 - Market / Purchasing Data Purchases

Erman_Ayday,Wireless_Headphones
Erman_Ayday,Gaming_Mouse
Wiam_Skakri,Wireless_Headphones
Wiam_Skakri,Smart_Light_Bulb
Kaleb_Kim,Gaming_Mouse
Kaleb_Kim,Mechanical_Keyboard
Kent_Manion,Mechanical_Keyboard
Kent_Manion,Portable_Monitor
Aidan_Bugayong,Mechanical_Keyboard
Aidan_Bugayong,Smart_Light_Bulb
Jonah_Lorenzo,Portable_Monitor
Jonah_Lorenzo,Noise-Cancelling_Earbuds
Carson_Whitehouse,Smart_Light_Bulb
Carson_Whitehouse,Wireless_Charger
Yuqiao_Xu,Noise-Cancelling_Earbuds
Yuqiao_Xu,Gaming_Mouse
Tahir_Shah,Mechanical_Keyboard
Tahir_Shah,Portable_Monitor
Nathan_Link,Wireless_Headphones
Nathan_Link,Smart_Light_Bulb
Zeien_Chase,USB-C_Hub
Zeien_Chase,Wireless_Headphones
John_Yoon,Noise-Cancelling_Earbuds
John_Yoon,Gaming_Mouse
Darin_Hall,Broken_Camera

Demo Cont.

Client2 - Proprietary Patient and Disease Data

Erman_Ayday,Goated_Teaching_Disorder
Wiam_Skakri,Flu
Wiam_Skakri,Cold
Kaleb_Kim,Flu
Kent_Manion,Covid
Aidan_Bugayong,Covid
Jonah_Lorenzo,Cold
Carson_Whitehouse,Flu
Carson_Whitehouse,Insomnia

Client3 - User and Show Data

Erman_Ayday,Stranger_Things
Erman_Ayday,Penguins_of_Madagascar
Wiam_Skakri,Stranger_Things
Wiam_Skakri,Black_Mirror
Kaleb_Kim,Penguins_of_Madagascar
Kaleb_Kim,The_Witcher
Kent_Manion,Black_Mirror
Kent_Manion,You
Aidan_Bugayong,The_Witcher
Aidan_Bugayong,Bridgerton
Jonah_Lorenzo,You
Jonah_Lorenzo,Wednesday
Carson_Whitehouse,Bridgerton
Carson_Whitehouse,The_Queen's_Gambit
Yuqiao_Xu,The_Crown
Yuqiao_Xu,Stranger_Things
Tahir_Shah,Penguins_of_Madagascar
Tahir_Shah,Black_Mirror
Nathan_Link,The_Witcher
Nathan_Link,You
Deethya_Makonahalli,Bridgerton
Deethya_Makonahalli,Wednesday
John_Yoon,The_Queen's_Gambit
John_Yoon,Stranger_Things
Darin_Hall,The_Barbie_Show

Good Query Type

Nodes have High Presence

Nodes have higher presence in either the server or graph set. This means that either the node has more than a single connection in either graph, in other words together one of the graphs has significant information on the node

CN and Jaccard Consistency

Our two chosen link prediction statistics show similar results, meaning higher likelihood of the existence of a link

Common Attribute

Server shares their graph attribute(s) with formatting, thus any queries to the server should include at least one node from an attribute(s) of the server.

Bad Query Type

Sparse Client Data

Client 2’s disease graph data was extremely sparse with only a single disease appearing 1-3 times, thus making it hard to draw significant findings from the data

Queries on Obscure Nodes

Some nodes in the server have significantly less information on them than others. As a result querying on these nodes will result in less information gain. Unfortunately, due to the server-client nature of our implementation these queries are inevitable

No Common Attributes

If a query to the server is on two nodes not in the server’s attribute(s) then the nodes will consequently not be in the set of nodes of the graph. Thus this query would result in the same as a local link prediction and the server query is pointless.

Project Challenges

Selecting a Compatible PSI Backend

- Most PSI libraries (including FastPSI) are Linux-only, require AVX-512, or lack Python bindings
- Ended up using openmined-psi since it runs on Linux/Unix systems with cardinality-only mode

Serializing PSI Objects Over HTTP

- The C++ PSI library produces protobuf objects that can't be sent as JSON directly
- Ended up base64-encoding each protobuf message and deserializing on the other side

The Server-Client Connection

- Potential firewall and network restrictions when allowing any given pair of clients to connect
- Ended up using localtunnel and bypassed their splash page

A Pair Query Vulnerability

- A malicious client could repeatedly submit pairs of nodes to discover aspects of the servers graph
- The server will still run the protocol as normal

Tradeoffs

Privacy vs. Efficiency

- The paper offers a heavier homomorphic version that hides all intermediate values…
- But we chose the lightweight variant that reveals intermediate sizes to Party 1 for speed

Accuracy vs. Efficiency

- Jaccard normalizes CN by total degree, giving a more meaningful 0–1 score
- Requires 5 PSI calls instead of 3 and Party 2 revealing degree sizes

Memory vs. Storage

- Current implementation is simple, such that sessions live in a dict but never garbage-collected
- Persistent storage is required for production but unnecessary for a local environment

Neighbor counting vs. Neighbor weighting

- CN/Jaccard treat neighbors equally, which is supported by the paper
- Adamic-Adar/Katz requires fundamentally different PSI primitives or multi-round protocols

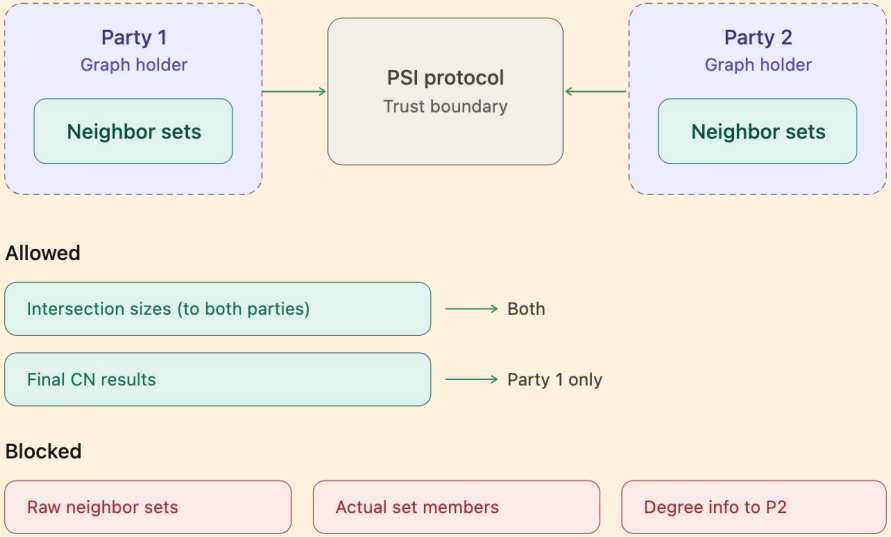
Threat Model - Semi Honest

Both parties are assumed to:

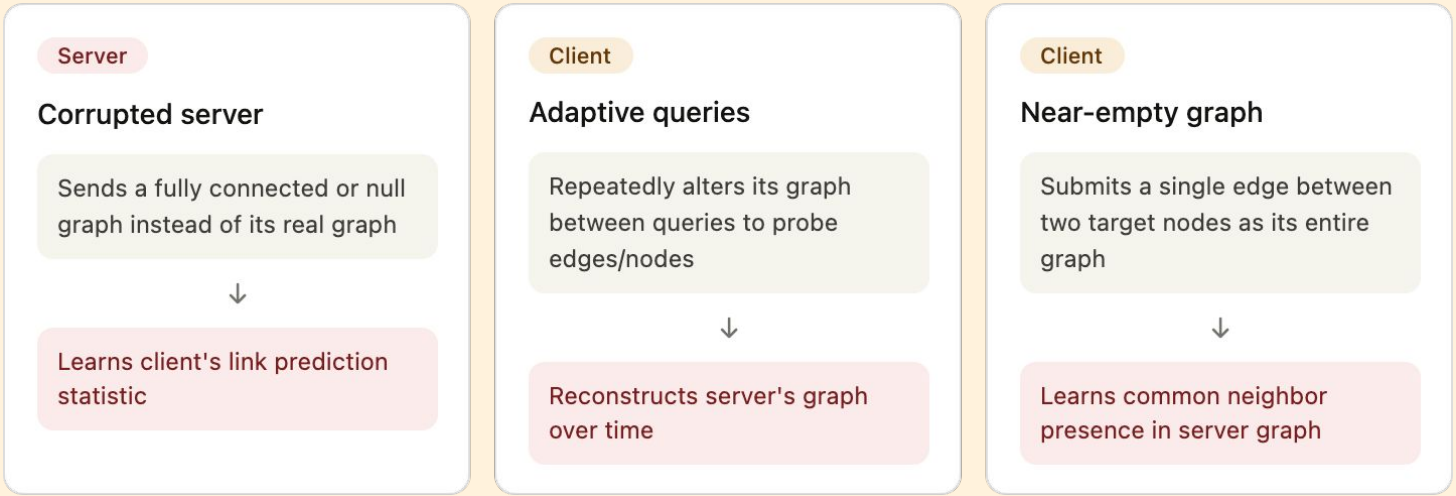
- Follow the protocol faithfully
  - They do not deviate
  - They do not send malformed input
  - They do not abort early
- Try to infer information from the protocol messages they legitimately receive

Security Goals:

- Party 1 must not learn party 2's raw neighbor sets for any node
- Party 2 must not learn party 1's raw neighbor sets for any node
- PSI calls only expose intersection sizes, never the actual set members
- Only party 1 learns the final CN results and party 2 learns nothing beyond the common public inputs



Threats and Attacks for Non-Honest Attackers



Server could deviate from the protocol by including a completely connected graph or a null graph, allowing the server to determine the selected link prediction statistic of the client

Client could continuously alter their graph after queries to determine the presence or absence of edges/nodes in the server graph, potentially leading to a reconstruction of the server's graph

Our system forces clients to query on nodes present in their own graph, thus a client could include a single edge connecting those nodes to determine the common neighbours presence in the server graph. Note this attack only works if both nodes are present in the server graph

System Limitations

How should known links be treated by the server?

- In order to prevent the client from partially reconstructing the servers graph using repeated queries, the server will not alert the client if the queried nodes are connected

Candidate Node Selection

- Current implementation requires client to manually select nodes to query, these nodes are not guaranteed to be represented in server’s graph

Results have Low Utility in Isolation

- CN and Jaccard measures are only useful in link prediction when computed for many nodes

System Compatibility

- openmined-psi not compatible with Windows systems, limiting to Unix like systems such as macOS and Linux

Future Work

Stronger Link Prediction Measures

- CN is the simplest measure of node connectivity, and a single result is meaningless in isolation
- Jaccard Index results in a slightly stronger statistic but is still relatively naive and simple
- Adamic-Adar measure could allow for stronger link prediction by accounting for hub nodes, however a privacy preserving implementation requires a different approach.

More advanced graph representations

- Current implementation uses a simple one-attribute graph
- Does not support weighted or directional graphs

Integration with database systems

- Current implementation requires user to manually specify the graph, or enter a CSV formatted file
- Integration with a database system could allow for clients and servers to easily perform queries on large data sets

Conclusion

- **Previous work**
  - (Chen, Laine & Rindal (2017) delivered a high-performance PSI but our goal was to implement Ling et. al (2025)
  - An ultra-fast PSI protocol built upon a novel, bucket-based Oblivious Key-Value Store (OKVS) and Vector Oblivious Linear Evaluation (VOLE)
- **Our contribution**
  - We succefully implemented Dirmiag et al. (2022) PPLP into a Python library to predict links while still preserving privacy
  - This technology allows users to take advantage of powerful link prediction tools without sacrificing their data privacy and keeping their connections anonymous

Thank you!
Question?